

3. Control Unit

Control Unit (CU) of processor translates machine instructions to the control signals in the proper sequence for the microoperations that implement them. The CU initiates series of sequential steps of microoperations. The control variables at any given time can be represented by string of 1's and 0's called a control word. Based on how the control signals are generated, Control Units are implemented in one of two ways

1. Hardwired Control
2. Microprogrammed Control

1. Hardwired Control Unit

A hardwired control unit generates control signals using fixed logic circuits. These circuits interpret instructions and produce the required control actions based on the contents of the Instruction Register (IR), the step counter, condition flags/codes, and external input signals such as interrupts.

The fixed logic circuitry is implemented using combinational circuits composed of decoders and encoders. The decoder interprets the instruction stored in the IR and produces intermediate signals. These signals, along with condition codes and external inputs, are fed into the encoder. The encoder then generates the appropriate control signals for instruction execution.

RISC architecture concept uses the hardwired control.

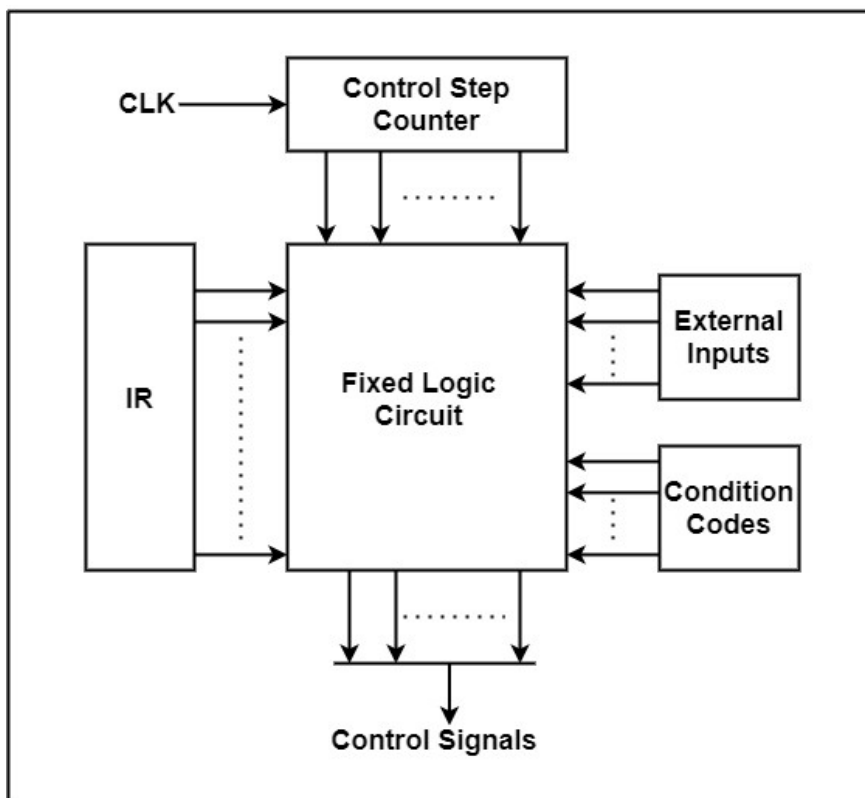


Fig. 1: Hardwired Control Unit

Advantages of a Hardwired Control Unit

- High speed of operation due to the use of combinational logic.
- Faster than a microprogrammed control unit.
- Can be optimized for minimal delay by careful gate design.
- Does not require control memory.

Disadvantages of a Hardwired Control Unit

- Design complexity increases as the number of control signals grows.
- Modifying control behavior is difficult because it requires hardware changes.
- Adding new instructions or features is complex and inflexible.

2. Microprogrammed Control Unit

2.1 Basic concept

A microprogrammed control unit is a control unit in which the binary control variables are stored in memory. The memory that forms part of the control unit is called control memory, and it may be implemented as read-only memory (ROM) or as writable control memory, the latter enabling dynamic microprogramming. A computer that uses a microprogrammed control unit contains two distinct memories:

- i. A main memory, which stores user programs and data
- ii. A control memory, which stores control information

2.2 Microinstruction, microprogram and control word

Each word in the control memory is known as a microinstruction. A sequence of microinstructions forms a microprogram, which generates all the control signals required to execute machine instructions correctly. A microinstruction consists of bits that initiate microoperations in the data-processing unit and bits that control the sequencing of addresses in control memory.

The collection of control variables that governs the execution of microoperations is called a control word. Each control word corresponds to a microinstruction and specifies one or more microoperations to be performed during a control step.

2.3 Wilkes Control

Maurice V. Wilkes of the Cambridge University Mathematical Laboratory first proposed the concept of a microprogrammed control unit in 1951. This approach introduced a systematic way to control processor operations using microinstructions stored in memory.

In Wilkes control, the complex sequential and combinational logic of a hardwired control unit is replaced by a simple control unit combined with a control memory. The control memory stores a microprogram, which is a sequence of microinstructions that define how each machine instruction is executed step by step.

Each microinstruction in Wilkes control consists of two main fields:

- a. Control field – specifies which control signals are to be activated to perform micro-operations.
- b. Address field – specifies the address of the next microinstruction to be executed.

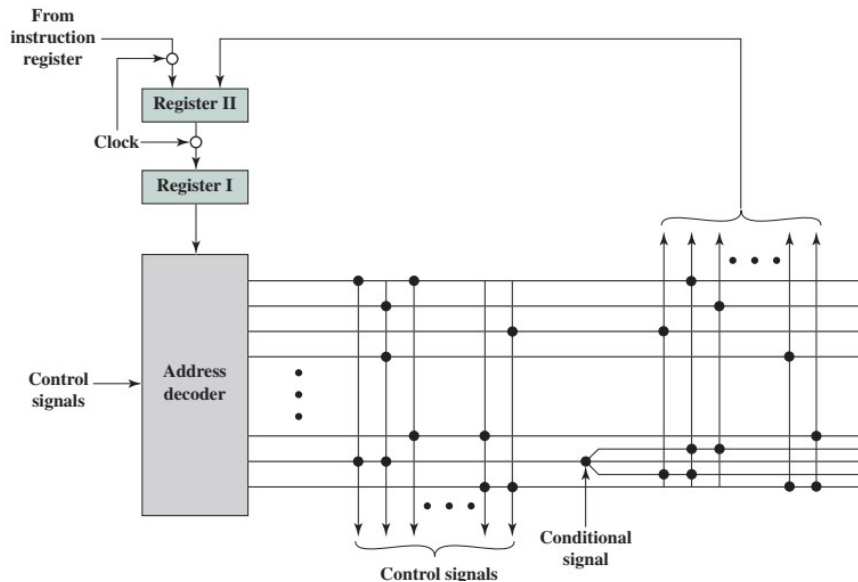


Fig. 2: Wilkes Microprogrammed Control Unit

Fig. 2 shows the configuration proposed by Wilkes. A control address register (CAR) (referred to as register I in the original description) holds the address of the current microinstruction. This address is decoded to activate the corresponding row in the control matrix. The resulting control signals execute the required micro-operations, while the next microinstruction address is loaded into an intermediate register (often called a control buffer register) before being transferred back to the CAR.

At the start of each control cycle, the address in the CAR is applied to a decoder, which is enabled by the clock pulse. During a machine cycle, only one row of the control matrix is activated.

The use of two registers is essential because the decoder is a combinational circuit. If only one register were used, its output could feed back into its input within the same cycle, potentially causing instability or unintended looping.

Advantages of microprogrammed control unit

- Simplifies the design of the control unit
- Cheaper and less error-prone than hardwired control
- Control logic is easy to modify by changing microprograms
- Uses simple decoding and sequencing hardware
- Well suited for complex instruction sets (CISC)

Disadvantages of microprogrammed control unit

- Slower than hardwired control units of similar technology
- Control signal generation requires memory access
- Not ideal for high-speed processors

Application

- Commonly used in CISC architectures
- RISC architectures usually employ hardwired control units due to simpler instruction formats and higher speed requirements

2.4 Microprogrammed control organization

In general, the microprogrammed controlled unit consist of (Fig. 3):

- i. Control address register (CAR): contains the address of the next microinstruction to be executed.
- ii. Control data register (CDR): contains the microinstruction to be executed
- iii. The sequencer: determines the next address of microinstruction within control memory.
- iv. Control memory: memory where microinstructions are stored.

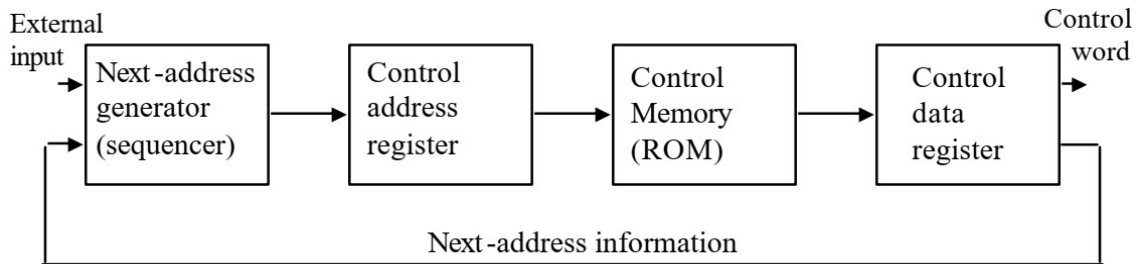


Fig. 3: Microprogrammed Control Organization

3 Address sequencing

The operation of the address sequencing in control unit can be explained as follows:

- An initial address is loaded into the control address register when the power is turned on in the computer.
- This address is usually the address of the first microinstruction that activates the instruction fetch routine.
- The control memory next must go through the routine that determine the effective address of the operand.
- The next step is to generate the microoperations that execute the instruction fetched from the memory.

The address sequencing capabilities required in control memory are:

- Incrementing the control address register
- Unconditional and conditional branching based on status bits
- Mapping instruction opcode bits to control memory addresses
- Support for subroutine call and return

Fig. 4 shows a block diagram of a control memroy and the associated hardware required for selecting the next microinstruction address.

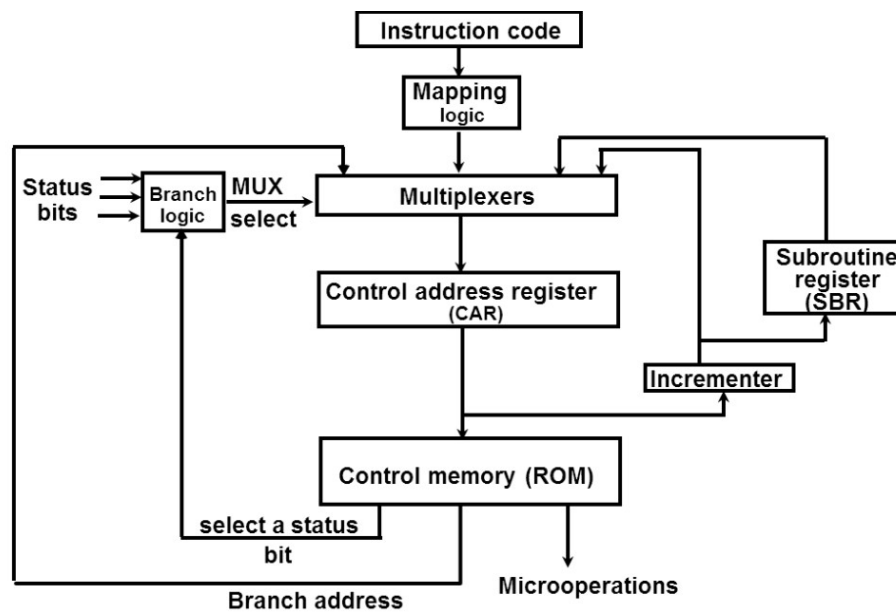


Fig. 4: Block diagram-Selection of address for control memory

The control address register (CAR) can receive the next address from four sources:

- i. Incrementer (next sequential microinstruction)
- ii. Branch address field in the microinstruction
- iii. Mapping logic (external address derived from opcode)
- iv. Subroutine return register (Return address for subroutine stored in SBR)

Conditional Branching

The branch logic (Fig. 4) provides decision-making capabilities in the control unit. The status conditions are special bits that provides parameter information, such as carry-out, sign, mode, etc. The status bits, together with the field in the microinstruction that specifies a branch address, control the conditional branch decisions generated in the branch logic.

The branch logic hardware can be implemented by mulitplexer.

- If the tested condition is true, branch to the specified address
- Otherwise, the address register is incremented.

Unconditional Branch

An unconditional branch in a microprogrammed control unit is implemented by forcing the multiplexer output to logic 1. This can be achieved by fixing one status bit permanently to 1 and selecting this input through the microinstruction. As a result, the branch address is loaded into the Control Address Register (CAR) irrespective of any condition.

Mapping of Instruction

A special type of branch exits when microinstruction specifies a branch to the first word in the control memory where a microprogram routine for an instruction is located. The status bits for this type of branch are the bits in the operation code part of the instruction.

One simple mapping process that converts the 4-bit operation code to a 7 bit address for the control memroy is shown in Fig. 5.

- Place a 0 in the most significant bit of the address.
- Transfer the four operation code bits

- Clear the two least significant bits of the control address register

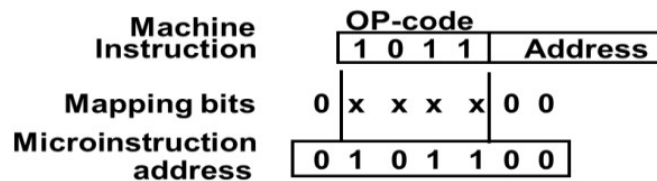


Fig. 5: Mapping rom instruction code to microinstruction address

This provides for each computer instruction (or machine instruction) a microprogram routine with a capacity of four micro instruction. If the routine needs more than four microinstructions, it can use addresses 1000000 through 1111111. If it uses fewer than four microinstructions, the unused memory locations would be available for other routines.

More flexible mapping method is direct mapping of instruction in control memory in which each machine instruction is mapped directly to a fixed starting address in the control memory. The opcode of the machine instruction is used directly as the address (or part of the address) of the first microinstruction in control memory as shown in Fig. 6.

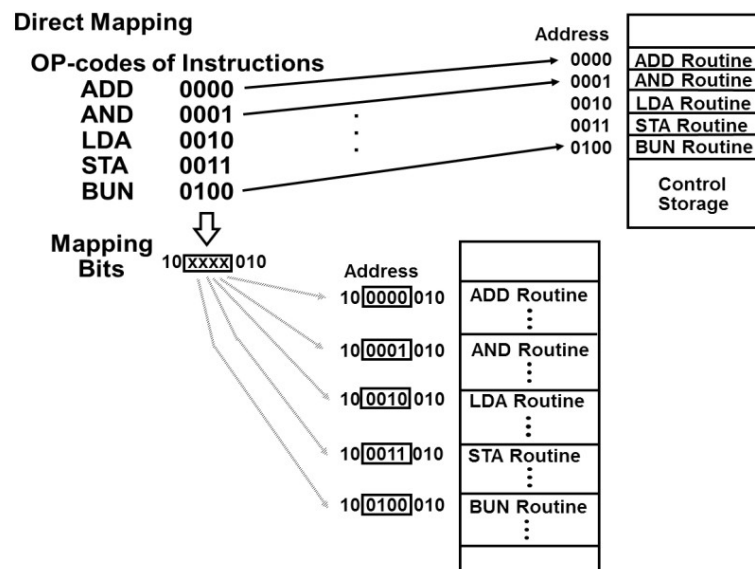


Fig. 6: Direct Mapping

Subroutine

Subroutines are microprogram segments that perform a specific task and can be called from multiple routines. They help reduce duplication of microinstructions, saving control memory space.

Commonly repeated operations (e.g., effective address computation for memory-reference instructions) are ideal candidates for subroutines. A subroutine can be invoked from any point within a microprogram.

Microprograms using subroutines must support:

- i. Saving the return address during a subroutine call
- ii. Restoring the return address when returning to the calling routine

The return address is typically obtained by incrementing the control address register (CAR) and storing it in a special subroutine register. On a subroutine return, the stored address is transferred back to the CAR.

The most effective way to manage return addresses is using a LIFO (Last-In, First-Out) stack, which supports nested subroutine calls.

4. Computer hardware configuration

Once the configuration of a computer and its microprogrammed control unit is established, the designer's task is to generate the microcode for the control memory. This microcode generation is called microprogramming. The block diagram of the computer is shown in Fig. 7.

It consist of:

- i. Two memory
 - A main memory for storing instructions and data
 - A control memory in control unit for storing the microprogram
- ii. Four registers associated with the processor unit
 - Program counter PC,
 - address register AR,
 - data register DR,
 - accumulator register AC
- iii. Two registers in control unit
 - control address register CAR and
 - a subroutine register SBR.

The control memory and its register are organized as a microprogrammed control unit. The transfer of information among the registers in the processor is done through multiplexers rather than a common bus.

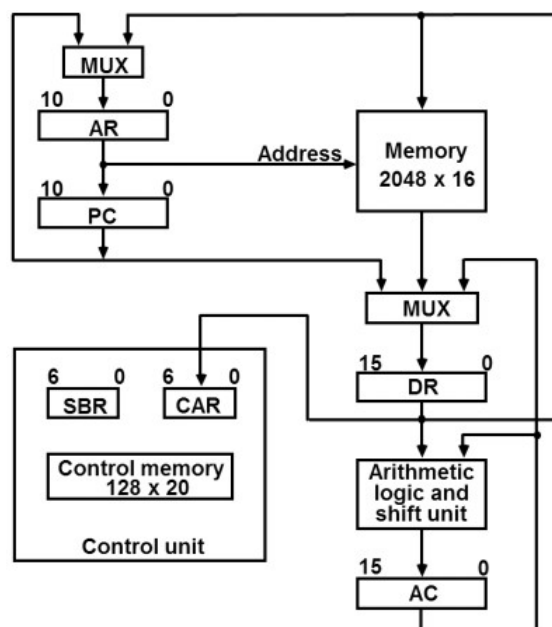
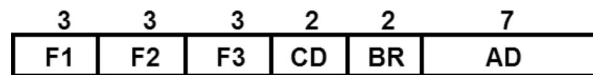


Fig. 7: Computer Hardware Configuration

Microinstruction format

The microinstruction format for the control memory is shown in Fig. 8. The 20 bits of the microinstruction are divided into four functional parts.

- i. The three fields F1, F2 and F3 specify microoperations
- ii. CD field selects status bit conditions
- iii. BR field selects the type of branch
- iv. AD field contains a branch address



F1, F2, F3: Microoperation fields

CD: Condition for branching

BR: Branch field

AD: Address field

Fig. 8: Microinstruction code format

Microoperations

The three bits in each microoperation fields are encoded to specify seven distinct microoperation as listed in table 1,2, and 3. However, no more than three microoperations can be chosen for a microinstruction, one from each field. If fewer than three microoperations are used, one or more of the fields will use the binary code 000 for no operation. Additionally, it is important to realize that two or more monflicting microoperations cannot be specified simulataneously. Example: 010 001 000

Table 1: Symbols and binary code for microinstruction fields, F1

F1	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC + DR$	ADD
010	$AC \leftarrow 0$	CLRAC
011	$AC \leftarrow AC + 1$	INCAC
100	$AC \leftarrow DR$	DRTAC
101	$AR \leftarrow DR(0-10)$	DRTAR
110	$AR \leftarrow PC$	PCTAR
111	$M[AR] \leftarrow DR$	WRITE

Table 2: Symbols and binary code for microinstruction fields, F2

F2	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC - DR$	SUB
010	$AC \leftarrow AC \vee DR$	OR
011	$AC \leftarrow AC \wedge DR$	AND
100	$DR \leftarrow M[AR]$	READ
101	$DR \leftarrow AC$	ACTDR
110	$DR \leftarrow DR + 1$	INCDR
111	$DR(0-10) \leftarrow PC$	PCTDR

Table 3: Symbols and binary code for microinstruction fields, F3

F3	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC \oplus DR$	XOR
010	$AC \leftarrow AC'$	COM
011	$AC \leftarrow shl AC$	SHL
100	$AC \leftarrow shr AC$	SHR
101	$PC \leftarrow PC + 1$	INCP
110	$PC \leftarrow AR$	ARTPC
111	Reserved	

The CD field consists of two bits which are encoded to specify four status bit conditions as listed in Table 4. The BR field consists of two bits. It is used, in conjunction with the address field AD, to choose the address of the next microinstruction.

- The jump and call operations depend on the value of the CD field.
- The two operations are identical except that a call microinstruction stores the return address in the subroutine register SBR.
- Note that the last two conditions in the BR field are independent of the values in the CD and AD fields.

Table 4: Symbols and binary code for microinstruction fields, CD and BR

CD	Condition	Symbol	Comments
00	Always = 1	U	Unconditional branch
01	DR(15)	I	Indirect address bit
10	AC(15)	S	Sign bit of AC
11	AC = 0	Z	Zero value in AC

BR	Symbol	Function
00	JMP	$CAR \leftarrow AD$ if condition = 1 $CAR \leftarrow CAR + 1$ if condition = 0
01	CALL	$CAR \leftarrow AD, SBR \leftarrow CAR + 1$ if condition = 1 $CAR \leftarrow CAR + 1$ if condition = 0
10	RET	$CAR \leftarrow SBR$ (Return from subroutine)
11	MAP	$CAR(2-5) \leftarrow DR(11-14), CAR(0,1,6) \leftarrow 0$

Fetch Cycle

The control memory has 128 words, each word contains 20 bits. To microprogram the control memory, it is necessary to determine the bit values of each of the 128 words. The first 64 words are to be occupied by the routines for 16 instructions, The last 64 words may be used for any other purpose.

The beginning of each instruction cycle is the fetch cycle, and causes an instruction to be fetched from memory. The fetch cycle consists of four microoperations that are executed in three timing steps as follow;

T1: $AR \leftarrow PC$

T2: $DR \leftarrow M[AR], PC \leftarrow PC+1$

T3: $AR \leftarrow DR(0-10), CAR(2-5) \leftarrow DR(11-14), CAR(0,1,6) \leftarrow 0$

The address of the instruction is transferred from PC to AR and instruction then read from memory to DR: Since no instruction register is available, instruction code remains in DR. The address part is transferred to AR and then control is transferred to one of 16 routines by mapping the operation code part of the instruction from DR to CAR.

5. Design of the control unit

The bits of the microinstruction are usually divided into fields, with each field defining a distinct, separate function. The various fields encountered in instruction formats provide:

- Control bits to initiate microoperations in the system
- Special bits to specify the way that the next address is to be evaluated
- An address field for branching

The number of control bits that initiate microoperations can be reduced by grouping mutually exclusive variables into fields by encoding the k bits in each field to provide 2^k microoperations. Each field requires a decoder to produce the corresponding control signals. The use of decoders:

- Reduces the size of the microinstruction bits
- Requires additional hardware external to the control memory
- Increases the delay time of the control signals

Fig. 9. shows the three decoders and some of the connections that must be made from their outputs. Outputs 5 or 6 of decoder F1 are connected to the load input of AR so that when either one of these outputs is active; information from the multiplexers is transferred to AR.

The transfer into AR occurs with a clock pulse transition only when output 5 (from DR (0-10) to AR i.e. DRTAR) or output 6 (from PC to AR i.e. PCTAR) of the decoder are active. The arithmetic logic shift unit can be designed instead of using gates to generate the control signals; it comes from the outputs of the decoders.

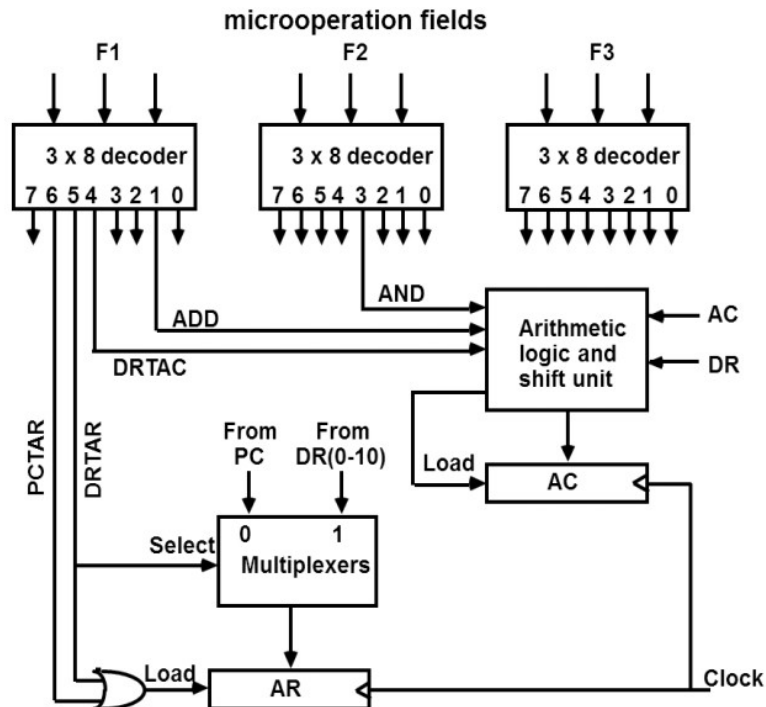


Fig. 9: Decoding of microoperation fields

Microprogram Sequencer

The basic components of a microprogrammed control unit are the control memory and the circuitry responsible for selecting the next microinstruction address. This address-selection circuitry is known as the microprogram sequencer.

A microprogram sequencer can be designed using digital logic to meet the requirements of a specific application. However, to ensure broad applicability, an integrated-circuit sequencer must have an internal organization that can be adapted to a wide range of systems. The primary function of a microprogram sequencer is to generate and supply the appropriate address to the control memory so that the next microinstruction can be fetched and executed.

The block diagram of a microprogram sequencer as shown in Fig. 10 illustrates its interaction with the control memory. The sequencer contains two multiplexers:

- The first multiplexer selects the next address from one of four possible sources and transfers it to the Control Address Register (CAR). The four sources are:
 - Incrementer (next sequential microinstruction)
 - Branch address field in the microinstruction
 - Mapping logic (external address derived from opcode)
 - Subroutine return register (Return address for subroutine stored in SBR)
- The second multiplexer selects a status bit specified by the microinstruction. The value of this selected status bit is then applied to the input logic circuit.

The contents of the CAR provide the address to the control memory. At the same time, the CAR contents are incremented and supplied as an input to the first multiplexer and stored in the Subroutine Register (SBR). Although the diagram shows a single SBR, practical microprogram sequencers typically use a stack of registers, usually four to eight levels deep. This stack supports push and pop operations, controlled by a stack pointer, to facilitate microprogram subroutine calls and returns.

The Condition (CD) field of the microinstruction selects a specific status bit through the second multiplexer. The resulting test value ($T = 0$ or 1), together with the two bits from the Branch (BR) field, is fed into the input logic circuit. Based on these inputs, the input logic circuit determines the type of sequencing operation to be performed, such as sequential execution, branching, or subroutine control.

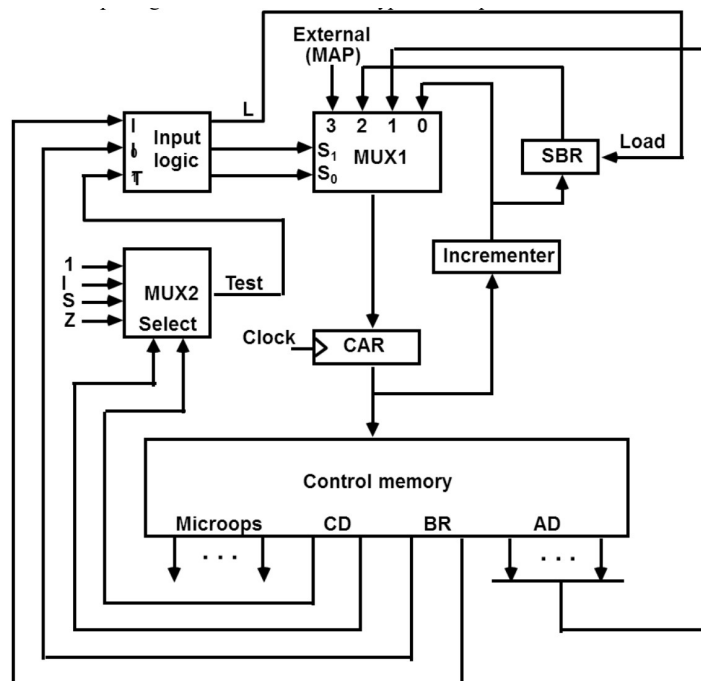


Fig. 10: Microprogram sequencer for a control memory

Design of Input Logic in microprogram sequencer

The input logic in a particular sequencer will determine the type of operations that are available in the unit. Typical sequencer operations are: increment, branch or jump, call and return from subroutine, load an external address, push or pop the stack, and other address sequencing operations. The truth table for the input logic circuit is shown in Table 5.

BR Field	Input			MUX 1		Load SBR
	I_1	I_0	T	S_1	S_0	L
0 0	0	0	0	0	0	0
0 0	0	0	1	0	1	0
0 1	0	1	0	0	0	0
0 1	0	1	1	0	1	1
1 0	1	0	X	1	0	0
1 1	1	1	X	1	1	0

Table 5: Input logic truth table for microprogram sequencer

The simplified boolean functions for the input logic circuit can be given as follow:

$$S_1 = I_1$$

$$S_0 = I_1 I_0 + I_1' T$$

$$L = I_1' I_0 T$$

The bit values for S1 and S0 are determined from the stated function and the path in the multiplexer that establishes the required transfer. It should be noted that the incrementer circuit in the sequencer diagram is not a counter constructed with flip-flops but rather a combinational circuit constructed with gates.

6. Microinstruction sequencing techniques

Techniques are grouped into three general categories based on the format of address information in the microinstruction:

- Two-address fields
- Single-address field
- Variable-format microinstructions

1. Two-address fields

Each microinstruction contains two address fields. A multiplexer selects either the opcode or one of the addresses for the Control Address Register (CAR). Branch logic determines the selection using flags and microinstruction bits.

Pros: Simple and straightforward.

Cons: Requires more bits per microinstruction → larger control memory.

2. Single-address field

Only one address field is included in the microinstruction. Options for the next address:

- Address field value
- Instruction register code
- Next sequential address

Address-selection signals determine which is used.

Pros: Reduces microinstruction size.

Cons: The single address field may remain unused often, causing coding inefficiency.

3. Variable-format microinstructions

Two different microinstruction formats indicated by a format-selection bit.

Format 1: Bits activate control signals; next address is sequential or from instruction register.

Format 2: Bits drive branch logic; remaining bits provide branch address.

Microinstruction sequencing: Address generation

Address generation techniques can be categorized as:

i. Explicit techniques

In explicit techniques, address is directly available in the microinstruction.

Examples:

- Two-address field: Two possible next addresses per microinstruction.
- Single-address field or variable format: Supports conditional and unconditional branches.

Conditional branches may depend on:

- ALU flags, such as Zero (Z), Carry (C), Overflow (V)
- Opcode or addressing mode bits
- Specific bits in a register (e.g., sign bit)
- Control unit status bits

ii. Implicit techniques

Additional logic is needed to generate the address.

Example: Mapping

The opcode of a machine instruction is mapped to a microinstruction address. Mapping occurs once per instruction cycle after instruction fetch

7 Microinstruction execution

A taxonomy of microinstructions

Microinstructions can be classified in a variety of ways

- Vertical/Horizontal
- Packed/Unpacked microprogramming
- Hard/soft
- Direct/Indirect encoding

i. Vertical/horizontal

- Related to instruction width
- Vertical: shorter (16–40 bits), more encoded, less direct control per bit.
- Horizontal: wider (40–100 bits), more direct control over hardware signals.

ii. packed/unpacked

- Related to degree of encoding
- Packed instructions encode more control functions per bit

iii. Hard/soft microprogramming

- Related to closeness to hardware
- Hard: fixed, typically in ROM, closely tied to hardware.
- Soft: more flexible, can be user-modified.

iv. Direct/Indirect encoding

- represents how control signals are represented in the microinstruction
- direct bit-to-signal vs. encoded/decode logic

Microinstructions Encoding

Microinstructions are divided into fields, each containing a code (a binary value) that, when decoded, activates one or more control signals.

When the microinstruction executes:

- Every field is decoded simultaneously.
- Each decoded field activates one or more control signals.

With N fields, N simultaneous actions can be specified, each activating certain control signals.

Field rules:

- Each control signal is activated by at most one field.
- Each control signal must be activatable by at least one field.
- Codes within a field are mutually exclusive, so only one action per field occurs at a time.

Within a single field of L bits:

- There are 2^L possible codes.
- Only one code is present at a time → the actions (control signal patterns) specified by different codes in the same field are mutually exclusive (only one can occur per cycle).

Two Approaches to Organizing Fields

1. Functional encoding

- Fields are grouped by function type performed in the machine.
- Example: One field controls sources for data transfer to the accumulator (e.g., different codes select different registers or buses as the source).
- Focuses on operations/functions (e.g., shift, add, logical, I/O).

2. Resource encoding

- Views the processor as a set of independent resources (e.g., ALU, memory interface, I/O unit, registers).
- Dedicates one field per resource (or group of related resources).
- Each field's codes control actions specific to that resource.

Another aspect: Direct vs. Indirect Encoding

Direct encoding:

- Each field directly maps (via decoding) to a specific control signal pattern.
- Simpler, usually single-level decoding → lower propagation delay.

Indirect encoding:

- One field determines the interpretation of another field (acts as a "mode" selector).
- Example: ALU supports 8 arithmetic ops + 8 shift ops.
 - 1-bit field selects "arithmetic" vs. "shift".
 - 3-bit field specifies which of the 8 operations in the selected category.
- Requires two levels of decoding → increases propagation delay (slower execution).

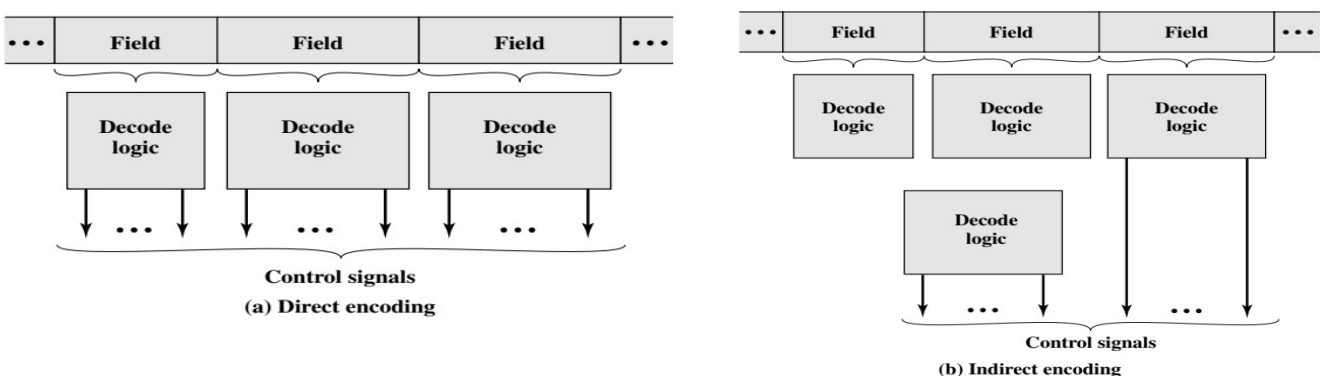


Fig. 11: Direct and indirect encoding

Alternative Microinstruction Formats: Vertical vs Horizontal

Highly vertical format (more encoded, narrower word):

- First 3 bits: Type of operation.
- Next 3 bits: Specific operation (within type).
- Last 2 bits: Select an internal register (e.g., PC, temp register for ALU).
- Compact, high encoding, resembles machine instructions.

Simple register transfers

0	0	0	0	0	0		
---	---	---	---	---	---	--	--

 $\text{MDR} \leftarrow \text{Register}$

0	0	0	0	0	1		
---	---	---	---	---	---	--	--

 $\text{Register} \leftarrow \text{MDR}$

0	0	0	0	1	0		
---	---	---	---	---	---	--	--

 $\text{MAR} \leftarrow \text{Register}$

Register
select

Memory operations

0	0	1	0	0	0		
---	---	---	---	---	---	--	--

Read

0	0	1	0	0	1		
---	---	---	---	---	---	--	--

Write

Special sequencing operations

0	1	0	0	0	0		
---	---	---	---	---	---	--	--

 $\text{CSAR} \leftarrow \text{Decoded MDR}$

0	1	0	0	0	1		
---	---	---	---	---	---	--	--

 $\text{CSAR} \leftarrow \text{Constant (in next)}$

0	1	0	0	1	0		
---	---	---	---	---	---	--	--

Skip

ALU operations

0	1	1	0	0	0		
---	---	---	---	---	---	--	--

 $\text{ACC} \leftarrow \text{ACC} + \text{Register}$

0	1	1	0	0	1		
---	---	---	---	---	---	--	--

 $\text{ACC} \leftarrow \text{ACC} - \text{Register}$

0	1	1	0	1	0		
---	---	---	---	---	---	--	--

 $\text{ACC} \leftarrow \text{Register}$

0	1	1	0	1	1		
---	---	---	---	---	---	--	--

 $\text{Register} \leftarrow \text{ACC}$

0	1	1	1	0	0		
---	---	---	---	---	---	--	--

 $\text{ACC} \leftarrow \text{Register} + 1$

Register
select

Fig 12: Vertical microinstruction format

Horizontal format (less encoded, wider word, still used encoding):

- Different functions appear in separate fields.
- Allows more parallelism (multiple independent actions per cycle).
- Still encodes within fields to save bits compared to fully unencoded (one-bit-per-signal).

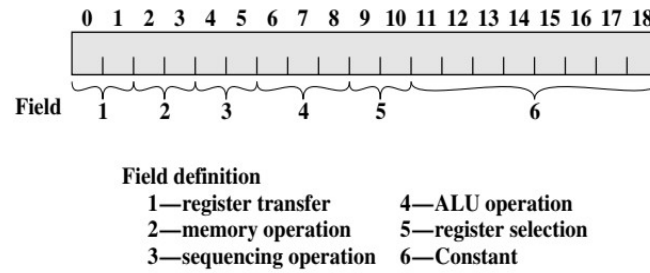


Fig. 13 Horizontal microinstruction format